

1. Plan de test unitaire

1.1. **Identification du test**

Ihm::CreationZone_v1

1.2. **Référence au module testé**

Méthode testé : string CreationZone(List<point> points)

Cette méthode appartient à la classe ihm et aura comme possible valeur de retour :

Valeur de retour	Description
resultat	indique comment s'est passé la création des zones

1.3. **Objectif du test**

Ce test aura pour but de tester la bonne création des zones par la méthode CreationZone en lui passant en paramètre des listes de point factice

1.4. **Procédure de test**

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point correspondant au cas de test mis en oeuvre
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

1.4.1. Valeurs de test

**Pour faire ces cas de test on se base sur une résolution d'écran de 1920x1080*

Nom du test	Description du vecteur test
TC_01	point = {(400; 500), (600; 800), (200; 300), (0; 100)}
TC_02	points = {(1920; 1080), (0; 1080), (0;0), (1920; 0)}
TC_03	points = {(1920; 1080), (0; 1080), (0;540), (1920; 540)}
TC_04	points = {(-400; 500), (600; 800), (200; 300), (0; 100)}
TC_05	points = (3000; 2800), (3000; 2500), (2500; 2500), (2500; 2800)
TC_06	points = (1920; 2800), (1920; 550), (0; 550), (0; 2800)
TC_07	points = {(400; 500), (600; 800), (200; 300)}
TC_08	points = {(400; 500), (600; 800), (200; 300), (0; 100), (850; 100)}
TC_09	points = {null, (600; 800), (200; 300), (0; 100)}

1.4.2. TC_01 : Cas Valide : une zone lambda

a. Initialisation :

points est une **liste de point contenant un point de coordonnées (400; 500), (600; 800), (200; 300), (0; 100)**

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point ayant pour coordonnées (400; 500), (600; 800), (200; 300), (0; 100)
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

c. Observation des résultat

Pour observer les resultat on va comparer le résultat renvoyé par la méthode testé avec les résultats attendu spécifier dans la partie 1.5 du plan de test avec la méthode **Assert.AreEqual()**

Le test est valide si Assert.AreEqual ne retourne aucune exception

1.4.3. TC_02 : Cas Limite : une zone qui fait la taille de l'écran

a. Initialisation :

points est une liste de point contenant un point de coordonnées (1920; 1080), (0; 1080), (0;0), (1920; 0)

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point ayant pour coordonnées (1920; 1080), (0; 1080), (0;0), (1920; 0)
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

c. Observation des résultat

Pour observer les resultat on va comparer le résultat renvoyé par la méthode testé avec les résultats attendu spécifier dans la partie 1.5 du plan de test avec la méthode **Assert.AreEqual()**

Le test est valide si Assert.AreEqual ne retourne aucune exception

1.4.4. TC_03 : Cas Limite 2: une zone qui fait la moitié de l'écran

a. Initialisation :

points est une liste de point contenant un point de coordonnées (1920; 1080), (0; 1080), (0;540), (1920; 540)

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point ayant pour coordonnées (1920; 1080), (0; 1080), (0;540), (1920; 540)
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

c. Observation des résultat

Pour observer les resultat on va comparer le résultat renvoyé par la méthode testé avec les résultats attendu spécifier dans la partie 1.5 du plan de test avec la méthode **Assert.AreEqual()**

Le test est valide si Assert.AreEqual **ne retourne aucune exception**

1.4.5. TC_04 : Cas Invalide : la zone contient une coordonnées négative

a. Initialisation :

points est une **liste de point contenant un point de coordonnées (-400; 500), (600; 800), (200; 300), (0; 100)**

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point ayant pour coordonnées (-400; 500), (600; 800), (200; 300), (0; 100)
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

c. Observation des résultat

Pour observer les résultats on va comparer l'exception retourné par la méthode testé et l'exception attendu a l'aide de la méthode **Assert.ThrowsException<>**

La méthode est valide si la méthode testé retourne une exception de type **ArgumentException**

1.4.6. TC_05 : Cas Invalide : une zone hors limite

a. Initialisation :

points est une liste de point contenant 4 points de coordonnées (3000; 2800), (3000; 2500), (2500; 2500), (2500; 2800)

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point ayant pour coordonnées (3000; 2800), (3000; 2500), (2500; 2500), (2500; 2800)
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

c. Observation des résultat

Pour observer les résultats on va comparer l'exception retourné par la méthode testé et l'exception attendu a l'aide de la méthode **Assert.ThrowsException<>**

La méthode est valide si la méthode testé retourne une exception de type **ArgumentException**

1.4.7. TC_06 : Cas Invalide : une zone dont la moitié est hors limite

a. Initialisation :

points est une liste de point contenant 4 points de coordonnées (3000; 2800), (1920; 550), (0; 550), (0; 2800)

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point ayant pour coordonnées (3000; 2800), (1920; 550), (0; 550), (0; 2800)
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

c. Observation des résultat

Pour observer les résultats on va comparer l'exception retourné par la méthode testé et l'exception attendu a l'aide de la méthode

Assert.ThrowsException<>

La méthode est valide si la méthode testé retourne une exception de type **ArgumentException**

1.4.8. TC_07 : Cas Invalide: 3 point lambda

a. Initialisation :

points est une liste de point **contenant 3 points de coordonnées (400; 500), (600; 800), (200; 300)**

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point ayant pour coordonnées (400; 500), (600; 800), (200; 300)
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

c. Observation des résultat

Pour observer les résultats on va comparer l'exception retourné par la méthode testé et l'exception attendu a l'aide de la méthode

Assert.ThrowsException<>

La méthode est valide si la méthode testé retourne une exception de type **ArgumentException**

1.4.9. TC_08 : Cas Valide : 5 points lambda

a. Initialisation :

points est une **liste de point contenant 5 points de coordonnées (400; 500), (600; 800), (200; 300), (0;100), (850;100)**

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point ayant pour coordonnées (400; 500), (600; 800), (200; 300), (0;100), (850;100)
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

c. Observation des résultat

Pour observer les resultat on va comparer le résultat renvoyé par la méthode testé avec les résultats attendu spécifier dans la partie 1.5 du plan de test avec la méthode **Assert.AreEqual()**

Le test est valide si Assert.AreEqual **ne retourne aucune exception**

1.4.10. TC_09 : Cas Null : 4 points dont un null

a. Initialisation :

points est une **liste de point contenant un point null et 3 point de coordonnées (600; 800), (200; 300), (0;100)**

b. Lancement

- Ouverture du projet de test
- Configuration d'une liste de points contenant un point null et 3 point de coordonnées (600; 800), (200; 300), (0;100)
- Passage de la variable en paramètre de la méthode testé
- Lancement des tests

c. Observation des résultat

Pour observer les résultats on va comparer l'exception retourné par la méthode testé et l'exception attendu a l'aide de la méthode **Assert.ThrowsException<>**

La méthode est valide si la méthode testé retourne une exception de type **ArgumentException**

1.5. *Résultat attendu*

Nom du test	Résultat attendu
TC_01	La méthode renvoie un message disant que la zone a bien été créer
TC_02	La méthode renvoie un message disant que la zone a bien été créer
TC_03	La méthode renvoie un message disant que la zone a bien été créer
TC_04	La méthode retourne une exception de type ArgumentException disant que les point d'une zone ne peuvent pas avoir de coordonnées négative
TC_05	La méthode retourne une exception de type ArgumentException disant que une partie de la zone où toute la zone se trouve en dehors de l'écran
TC_06	La méthode retourne une exception de type ArgumentException disant que une partie de la zone où toute la zone se trouve en dehors de l'écran
TC_07	La méthode retourne une exception de type ArgumentException disant que le nombre de points fourni est insuffisant pour créer une zones
TC_08	La méthode renvoie un message disant que la zone a bien été créer (dans ce cas de test le 5 point ne sera pas pris en compte)
TC_09	La méthode retourne une exception de type ArgumentNullException disant que l'un des points est null

1.6. *Moyen à mettre en oeuvre*

Moyen pour mettre en oeuvre les tests :

- PC portable/fixe avec Windows 10/11 d'installé dessus
- Visual studio configurer pour pouvoir exécuter des projet C# et des projet MS Test
- Un écran (si pc fixe)

2. Procès Verbal

Élément testé	Ihm::CreationZone			
Objectif du test	Ce test aura pour but de tester la bonne création des zones par la méthode CreationZone en lui passant en paramètre des listes de point factice			
Nom du testeur	BELAIRE Kévin		Date	08/04/2026
Moyen mis en oeuvre	Logiciel : Visual Studio 2022	Matériel : PC		Outil de développement : Framework .NET, Framework MS_Test
Procédure du test : - Ouverture du projet de test - Configuration d'une liste de points contenant un point correspondant au cas de test mis en oeuvre - Passage de la variable en paramètre de la méthode testé - Lancement des tests				
Id	Description du vecteur test	Résultat attendu	Résultat obtenu	Validation (O/N)
TC_01	point = {(400; 500), (600; 800), (200; 300), (0; 100)}	La methode renvoie un message disant que la zone a bien été créer		
TC_02	points = {(1920; 1080), (0; 1080), (0;0), (1920; 0)}	La methode renvoie un message disant que la zone a bien été créer		
TC_03	points = {(1920; 1080), (0; 1080), (0;540), (1920; 540)}	La methode renvoie un message disant que la zone a bien été créer		
TC_04	points = {(-400; 500), (600; 800), (200; 300), (0; 100)}	La methode retourne une exception de type ArgumentException on disant que les point d'une zone ne peuvent pas avoir de coordonnées négative		
TC_05	points = (3000; 2800), (3000; 2500), (2500; 2500), (2500; 2800)	La methode retourne une exception de type ArgumentException on disant que une partie de la zone ou toute la zone se trouve en dehors de l'ecran		
TC_06	points = (1920; 2800), (1920; 550), (0; 550), (0; 2800)	La methode retourne une exception de type ArgumentException		

		on disant que une partie de la zone ou toute la zone se trouve en dehors de l'ecran		
TC_07	points = {(400; 500), (600; 800), (200; 300)}	La methode retourne une exception de type ArgumentException on disant que le nombre de points fournit est insuffisant pour créer une zones		
TC_08	points = {(400; 500), (600; 800), (200; 300), (0; 100), (850; 100)}	La methode renvoie un message disant que la zone a bien été créer (dans ce cas de test le 5 point ne sera pas pris en compte)		
TC_09	points = {null, (600; 800), (200; 300), (0; 100)}	La methode retourne une exception de type ArgumentNullException disant que l'un des points est null		